

Android Framework 16

심화 교육

4일 과정 (32시간) | 시스템 개발자를 위한 고급 과정

실습 환경: Windows 11 + Android Studio Emulator (API 36)

Binder IPC | Framework Core | System UI | APEX

실습 환경 구성

Windows 11

Host OS
Android Studio Ladybug+
최소 RAM 16GB 권장
Hyper-V / WHPX 활성화

Android Studio

SDK: Android 16 (API 36)
Emulator: x86_64 이미지
Google APIs 포함 이미지
ADB & Logcat 연동

AOSP 소스 (참조용)

android.googlesource.com
Android 16 브랜치
WSL2에서 repo sync 가능
소스 리딩 중심 학습

디버깅 도구

adb shell / dumpsys
Android Profiler
Layout Inspector
Perfetto

교육 일정 개요

Day 1 (8h)

Ch 0~3

Android 16 아키텍처 & Binder IPC 기초/실습

Day 2 (8h)

Ch 4~6

Binder 심화 분석 & 커스텀 System Service

Day 3 (8h)

Ch 7~10

Framework Core & Overlay & Jetpack

Day 4 (8h)

Ch 11~13

System UI 커스터마이징 & APEX 모듈 업데이트

DAY 1

Android 16 아키텍처 & Binder IPC 기초

8시간 | Chapter 0~3



CHAPTER 0

Android 시스템 아키텍처 개요

Android 16(API 36) 계층 구조와 핵심 변경사항

Android 시스템 아키텍처 계층도

Applications (System Apps / User Apps)

Java API Framework (AMS, WMS, PMS, ...)

Native C/C++ Libraries | Android Runtime (ART)

Hardware Abstraction Layer (AIDL HAL)

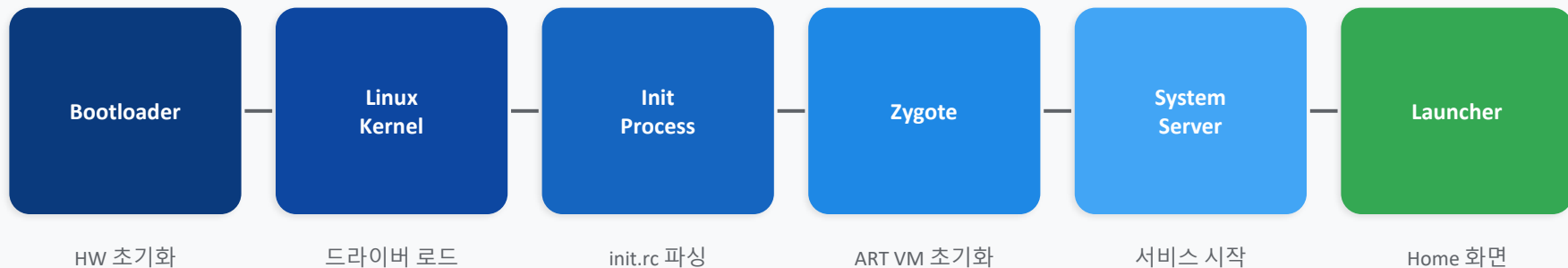
Linux Kernel (Binder, memfd, LMK, ...)

Android 16 주요 변경사항

⚠ 핵심 포인트 — 반드시 숙지하세요!

- 1 AIDL HAL 기본 — HIDL deprecated (신규 HAL은 AIDL 필수)
- 2 Privacy Sandbox 강화 — SDK Runtime 정식 적용
- 3 Predictive Back 제스처 API 확장
- 4 Health Connect 플랫폼 통합 (Jetpack → Framework 승격)
- 5 16KB Page Size 지원 확대 — ARM 성능 최적화

Android 부팅 시퀀스



💡 Zygote fork()로 앱 프로세스 생성 — COW(Copy-on-Write)로 메모리 효율적 공유

핵심 시스템 컴포넌트

Zygote

모든 앱의 부모 프로세스
fork()로 빠른 생성
ART VM + 클래스 사전 로딩
64bit/32bit 듀얼

System Server

AMS, PMS, WMS 등 핵심 서비스
첫 fork 프로세스
Binder IPC 통신
서비스 모듈화 확대

SurfaceFlinger

화면 합성 엔진
HWC 연동
VSync 기반 프레임 관리
Layer 기반 합성

ServiceManager

서비스 등록소 (handle 0)
이름 → Binder 참조 매핑
AIDL 기반 (Android 11+)
HAL 서비스 통합



현업 트러블슈팅: 부팅 및 시스템 초기화

✗ **Zygote crash loop** — 앱이 전혀 실행되지 않고 부팅 화면 반복

✓ logcat -b main -s Zygote / dmesg 확인 → SELinux 정책 또는 preloaded-classes 충돌

✗ **System Server ANR** — 부팅 후 30초 이상 응답 없음

✓ dumpsys activity → watchdog trace 확인, /data/anr/traces.txt 분석

✗ **에뮬레이터 부팅 실패** — WHPX/Hyper-V 충돌

✓ Windows 기능 → Hyper-V + WHPX 모두 활성화 확인, HAXM 제거

✗ **ART dex2oat 실패** — 앱 설치 시 `INSTALL_FAILED_DEXOPT`

✓ dalvik.vm.dex2oat-Xms 값 조정, 캐시 초기화: adb shell pm compile --reset

Chapter 0 연습문제

Q1 Android 16에서 HIDL 대신 AIDL HAL을 사용하는 이유를 설명하시오.

Q2 Zygote의 fork() + COW가 메모리 효율에 미치는 영향을 설명하시오.

Q3 System Server 시작 후 Launcher까지의 과정을 설명하시오.

Q4 에뮬레이터 부팅 실패 시 디버깅 절차를 단계별로 설명하시오.

CHAPTER 1

Binder 개념 및 IPC 이해

Android 핵심 IPC 메커니즘 심층 분석

전통적 IPC vs Binder IPC

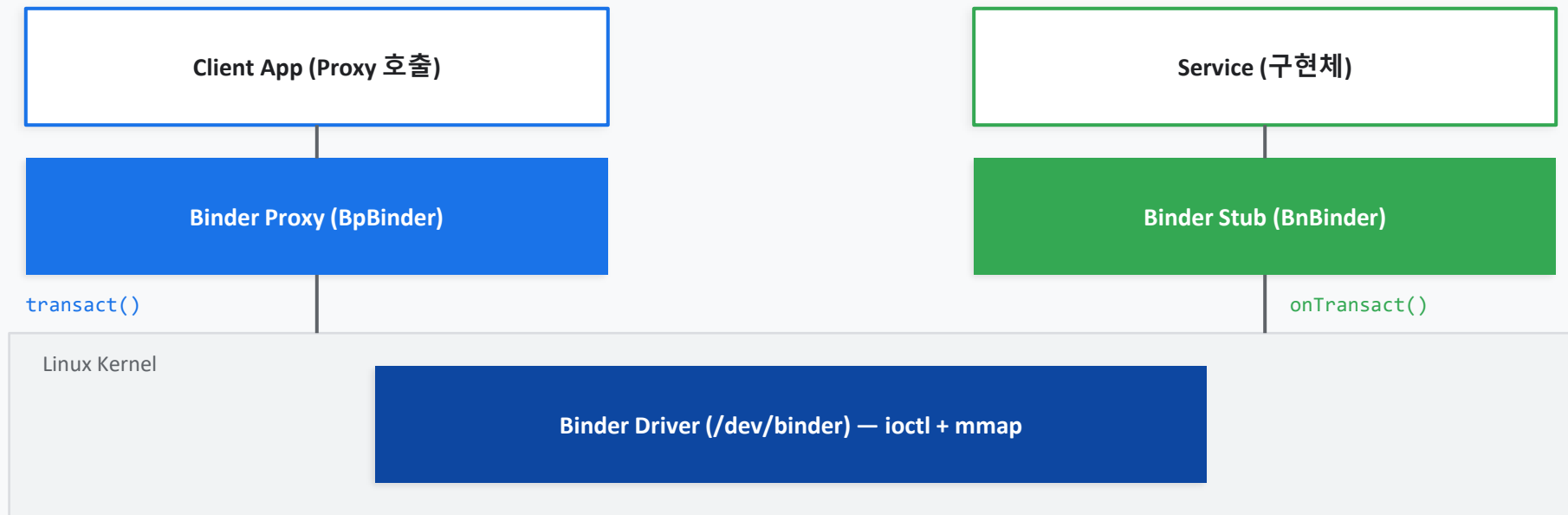
전통적 IPC

- Pipe: 단방향, 부모-자식 한정
- Socket: 범용적이거나 2회 복사
- Shared Memory: 빠르지만 동기화 복잡
- 공통: 호출자 신원 검증 어려움

Binder IPC

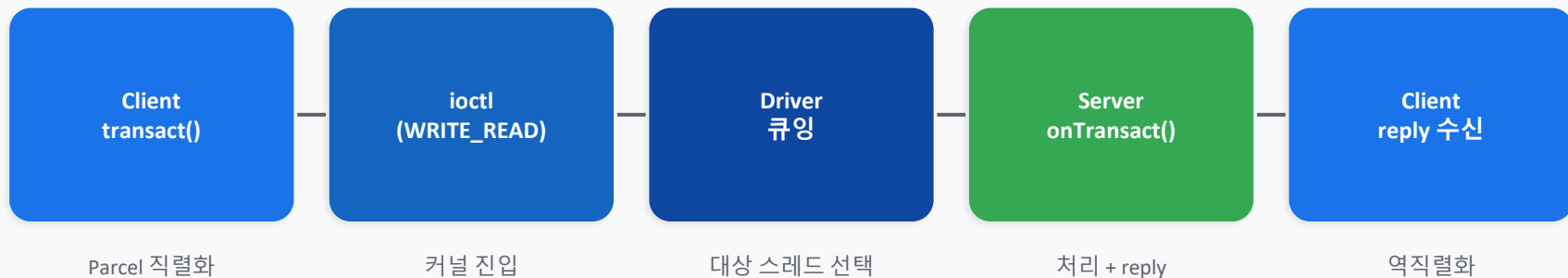
- 단 1회 복사 (mmap 활용)
- 커널 UID/PID 보장 → 위변조 불가
- 객체지향 RPC (메서드 호출)
- 참조 카운팅 + DeathRecipient

Binder 아키텍처 개요



💡 mmap으로 커널 버퍼와 서버 주소 공간을 동일 물리 페이지에 매핑 → 서버 측 zero-copy 수신

Binder Transaction 흐름 (동기 호출)



💡 FLAG_ONeway: 비동기 호출 — Client 즉시 반환 (콜백 패턴 필요)

ServiceManager 상세 동작

- Context Manager — 특수 handle 0에 등록된 글로벌 서비스
- `addService(name, IBinder)`: 시스템 서비스 등록 (시스템 권한 필요)
- `getService(name)`: Binder Proxy 반환 (서비스 미등록 시 블로킹)
- `checkService(name)`: 논블로킹 버전 — null 반환 가능
- Android 16: AIDL ServiceManager + VINTF 서비스 통합 관리
- **`waitForService()` 사용 시 timeout 설정 필수 — 무한 대기 위험**



현업 트러블슈팅: Binder IPC 오류

✗ **TransactionTooLargeException** — Binder 트랜잭션 1MB 초과

✓ 큰 데이터는 `ContentProvider/File`로 전달, `Bitmap`은 `ashmem(memfd)` 사용

✗ **DeadObjectException** — 서비스 프로세스 죽음

✓ `linkToDeath()`로 사전 등록, try-catch에서 재연결 로직 구현

✗ **SecurityException: Not allowed to bind** — SELinux 거부

✓ `logcat | grep avc:denied`로 정책 확인 → `sepolicy`에 `binder_call` 규칙 추가

✗ 서비스 `getService()` 무한 블로킹 — 부팅 순서 문제

✓ `checkService()` + 폴링 또는 `waitForService(timeout)` 사용



디버깅 가이드: Binder 상태 확인

```
# 시스템 서비스 목록
$ adb shell service list

# Binder 상태 (에뮬레이터 root)
$ adb root
$ adb shell cat /sys/kernel/debug/binder/state
$ adb shell cat /sys/kernel/debug/binder/stats
$ adb shell cat /sys/kernel/debug/binder/failed_transaction_log

# 특정 프로세스 Binder 정보
$ adb shell cat /sys/kernel/debug/binder/proc/<pid>

# Perfetto로 Binder 트랜잭션 프로파일링 (Android 16)
$ adb shell perfetto --config :test --out /data/misc/perfetto-traces/trace
```



Windows에서 에뮬레이터 사용 시 adb root가 Google APIs 이미지에서 기본 지원됨

Chapter 1 연습문제

Q1 TransactionTooLargeException의 발생 원인과 해결 방법을 3가지 설명하시오.

Q2 mmap을 통한 1회 복사 원리를 물리 페이지 관점에서 설명하시오.

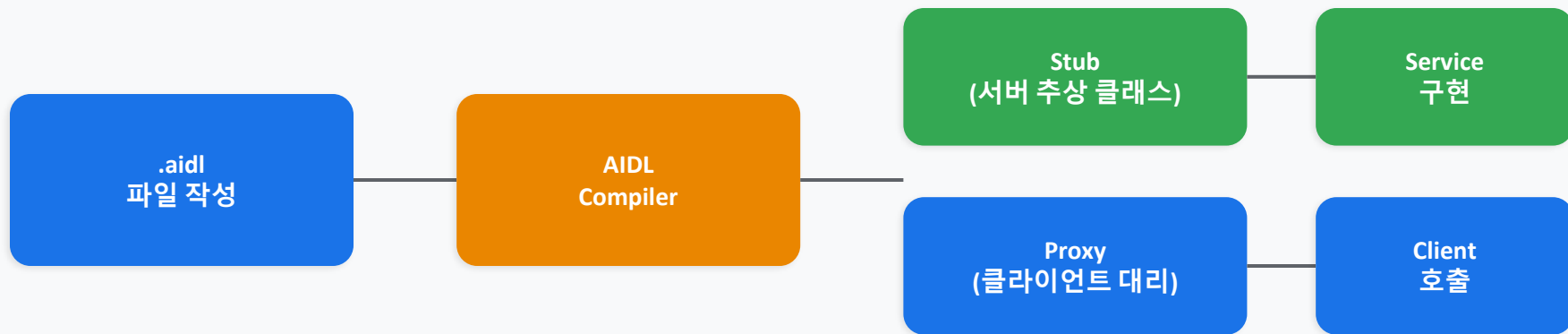
Q3 FLAG_ONeway 사용 시 주의사항과 콜백 패턴을 설명하시오.

CHAPTER 2

Binder 실습 (AIDL 기반 IPC)

Android Studio에서 AIDL 서비스 구현

AIDL 컴파일 과정



💡 AIDL 파일 하나로 Stub + Proxy 자동 생성 → 개발자는 Stub 구현과 Client 호출만 작성

AIDL 인터페이스 정의 (핵심부)

```
interface ICalculatorService {  
    int add(int a, int b);           // 동기  
    oneway void calcAsync(          // 비동기  
        int a, int b, ICalcCallback cb);  
}  
// ★ 커스텀 Parcelable 사용 시  
parcelable CalcResult; // 별도 .aidl 선언 필수
```

💡 in(기본)/out/inout: 파라미터 방향 → out은 빈 Parcel 전송 후 결과만 받음 (성능 차이 큼)

서비스 구현 – 핵심 보안 패턴

```
private final ICalculatorService.Stub mBinder =  
    new ICalculatorService.Stub() {  
        @Override public int add(int a, int b) {  
            // ★ 호출자 확인  
            int uid = Binder.getCallingUid();  
  
            // ★ 시스템 권한으로 작업 시 필수!  
            long token = Binder.clearCallingIdentity();  
            try { return a + b; }  
            finally { Binder.restoreCallingIdentity(token); }  
        }  
    };
```

💡 `clearCallingIdentity()`: 시스템 서비스가 자신의 UID로 다른 서비스 호출 시 반드시 사용

클라이언트 연결 – 핵심 패턴

```
// IBinder → Proxy 변환
mService = ICalculatorService.Stub.asInterface(service);
// ★ 같은 프로세스: Stub 직접 반환 (IPC 없음)
// ★ 다른 프로세스: Proxy 객체 생성 (IPC 발생)

try { int r = mService.add(10, 20); } // Binder IPC!
catch (RemoteException e) { /* 서비스 죽음 */ }

// ★ 서비스 사망 감지
service.linkToDeath(() -> mService = null, 0);
```

💡 linkToDeath 콜백은 Binder Thread에서 호출됨 — UI 작업 시 Handler 전환 필요



현업 트러블슈팅: AIDL 개발 흔한 오류

✗ AIDL 파일이 빌드에 포함되지 않음 — 인터페이스 클래스 생성 안됨

✓ src/main/aidl/ 디렉토리에 패키지 구조와 함께 배치, build.gradle에 aidl sourceSet 확인

✗ `ClassCastException: Stub.asInterface()` 반환 타입 불일치

✓ 서버/클라이언트 양쪽 AIDL 파일의 패키지명과 메서드 시그니처가 완전 동일해야 함

✗ `RemoteException: oneway` 메서드에서 예외 전달 안됨

✓ oneway는 비동기 → 서버 예외가 클라이언트에 전달 불가, 콜백 인터페이스로 에러 전달

✗ `Parcelable` 직렬화 순서 불일치 — 데이터 깨짐

✓ `writeToParcel()`과 `createFromParcel()`의 read/write 순서가 정확히 일치해야 함



실습: AIDL Calculator 서비스 (Android Studio)

1. New Project → Empty Activity (API 36 에뮬레이터 타겟)
2. src/main/aidl/ 예 ICalculatorService.aidl 작성
3. CalculatorService: Stub 구현 + clearCallingIdentity 패턴 적용
4. AndroidManifest.xml: <service exported="true"> 등록
5. ClientActivity: bindService() → asInterface() → 메서드 호출
6. linkToDeath() 등록 → 서비스 프로세스 kill 테스트
7. adb shell dumpsys activity services | grep Calculator 확인

Chapter 2 연습문제

- Q1** Stub.asInterface()의 내부 동작을 같은/다른 프로세스 경우로 나누어 설명하시오.
- Q2** AIDL Parcelable 직렬화 순서 불일치 시 발생하는 문제와 디버깅 방법을 설명하시오.
- Q3** clearCallingIdentity()가 필요한 구체적 시나리오를 3가지 제시하시오.

CHAPTER 3

간단한 IPC 서비스 개발

Messenger, Content Provider, Bound Service 패턴

Messenger vs AIDL 비교

Messenger

- Handler 기반 단방향 메시지
- 단일 스레드 → 자동 스레드 안전
- 구현 간편 (AIDL 불필요)
- Message + Bundle 전달
- 단순 명령 전달에 적합

AIDL

- 양방향 메서드 호출 (RPC)
- 멀티 Binder Thread Pool
- 강타입 인터페이스
- Parcelable 커스텀 타입
- 복잡한 API에 적합

Content Provider — Binder 기반 데이터 공유

- URI 기반 접근: content://authority/path/id
- CRUD: query(), insert(), update(), delete()
- 내부적으로 Binder IPC 사용 — call()도 Binder Transaction
- CursorWindow: 공유 메모리를 통한 대량 데이터 전달
- FileProvider: 파일 공유 표준 (Android 7.0+, FileUriExposedException 방지)
- ⚠ **CursorWindow 1MB 제한 — 대량 쿼리 시 페이징 필수**



현업 트러블슈팅: IPC 서비스 개발 오류

✗ `android.os.FileUriExposedException` — `file://` URI 직접 전달

✓ FileProvider로 `content://` URI 생성, `grantUriPermission()` 호출

✗ `ContentProvider query()` 에서 ANR — 메인 스레드 블로킹

✓ `CursorLoader` 또는 `ContentResolver.query()` 를 IO 스레드에서 호출

✗ `Messenger` 응답 수신 불가 — `replyTo` 미설정

✓ `Message.replyTo`에 클라이언트 `Messenger` 설정 후 `send()`

✗ `Process` 분리 시 `Application.onCreate()` 중복 실행

✓ 프로세스 이름 체크: `getProcessName()`으로 분기 처리

Chapter 3 연습문제

Q1 Content Provider가 Binder를 사용하는 방식과 CursorWindow의 역할을 설명하시오.

Q2 android:process 분리 시 주의사항 3가지를 설명하시오.

Q3 FileNotFoundException의 발생 원인과 해결 방법을 설명하시오.

DAY 2

Binder 심화 분석 & 커스텀 System Service

8시간 | Chapter 4~6

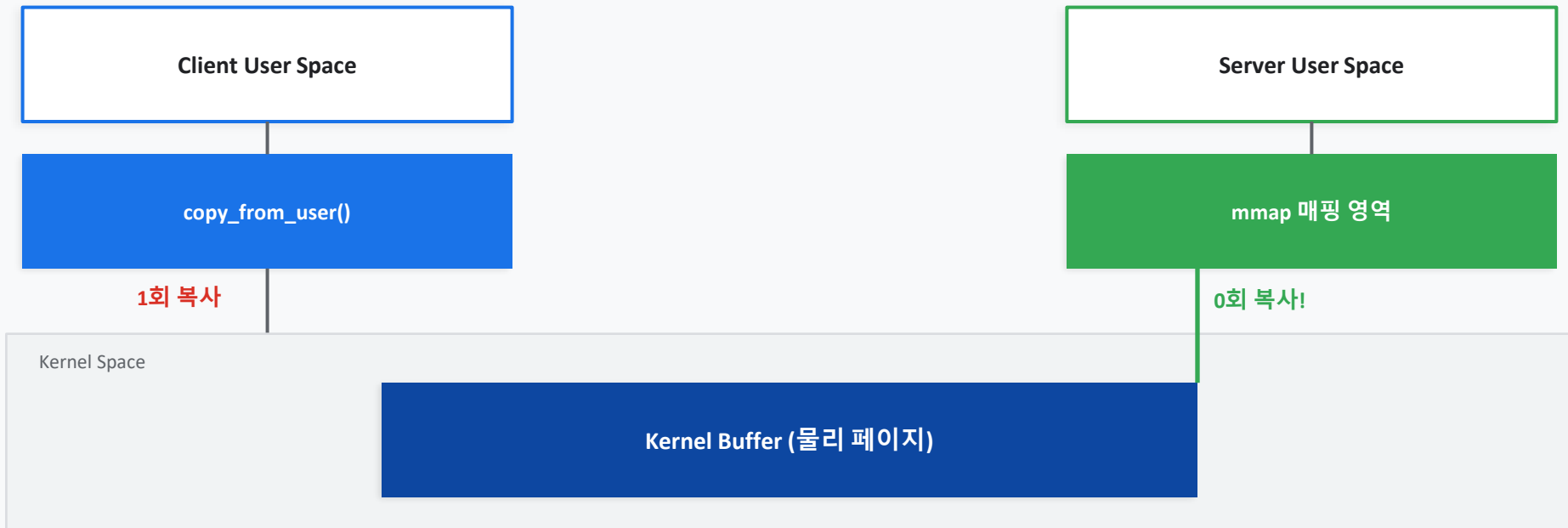


CHAPTER 4

Binder 심화 분석

커널 레벨 Binder Driver 내부 동작

Binder mmap 메모리 매핑 (Zero-Copy)



△ Kernel Buffer와 Server mmap 영역이 동일 물리 페이지 → 서버는 추가 복사 없이 읽기 가능

Binder Thread Pool 상세 동작

- 프로세스당 기본 최대 15+1 Binder Thread
- Main Binder Thread: looper — 자동 생성
- BR_SPAWN_LOOPER: 추가 스레드 동적 생성 트리거
- 모든 스레드 busy → 트랜잭션 대기열 큐잉
- ⚠ 데드락: $A \rightarrow B \rightarrow A$ 순환 동기 호출 시 스레드 고갈!
- 완화: oneway 사용 / 최대 스레드 수 증가 / 호출 방향 단일화
- 우선순위 상속: 호출자의 스케줄링 우선순위를 서버에 전파

Binder 트랜잭션 내부 구조

- binder_transaction: 송신자, 수신자, 데이터 버퍼, 오프셋 배열
- Flat Binder Object: Parcel 내 IBinder를 커널이 handle ↔ node 변환
- Scatter-Gather: 대용량 전송 시 효율적 버퍼 관리
- BR_TRANSACTION_COMPLETE: 전송 완료 알림 (oneway는 여기서 반환)
- BC_REPLY: 서버가 결과 반환 시 사용
- BR_DEAD_BINDER: 서버 프로세스 사망 → DeathRecipient 호출
- **binder_transaction 버퍼 최대 크기: ~1MB — 초과 시 TransactionTooLargeException**



현업 트러블슈팅: Binder 심화 오류

✖ Binder Thread 고갈 — 앱/시스템 서비스 응답 없음 (ANR)

✓ binder/state에서 thread 수 확인, oneway 전환 또는 BINDER_SET_MAX_THREADS 증가

✖ FAILED BINDER TRANSACTION — /dev/binder 버퍼 부족

✓ failed_transaction_log 확인, 대용량 데이터는 ashmem/FD 전달로 변경

✖ Priority Inversion — 낮은 우선순위 스레드가 Binder 잠금 보유

✓ Android 16에서 우선순위 상속 활용, real-time 스레드 사용 시 주의



디버깅 가이드: Binder 성능 프로파일링

```
# Binder 트랜잭션 시간 측정 (Perfetto)
$ adb shell perfetto -c - --txt <<EOF
buffers: { size_kb: 32768 }
data_sources: { config {
    name: "linux.ftrace"
    ftrace_config {
        ftrace_events: "binder/binder_transaction"
        ftrace_events: "binder/binder_transaction_received"
    }
}}
EOF

# systrace로 Binder 분석 (레거시)
$ python systrace.py -t 5 binder_driver -o trace.html

# 특정 프로세스 Binder 통계
$ adb shell cat /sys/kernel/debug/binder/proc/$(pidof system_server)

# Binder 호출 횟수 확인
$ adb shell dumpsys binder_calls_stats
```



Perfetto: Android 16 기본 트레이싱 도구 — systrace보다 상세한 분석 가능

Chapter 4 연습문제

Q1 Binder mmap zero-copy 원리를 물리 페이지 관점에서 설명하시오.

Q2 Binder Thread Pool 데드락 시나리오와 해결 방법을 설명하시오.

Q3 Perfetto로 Binder 성능 이슈를 분석하는 절차를 설명하시오.

CHAPTER 5

Native Binder vs Java Binder, 보안

C++ Binder 관계 및 Binder 보안 메커니즘

Java → Native Binder 호출 스택

App: mService.add(1, 2)

ICalcService.Stub.Proxy.add() → Parcel 직렬화

android.os.BinderProxy.transact() [Java Framework]

JNI: android_os_BinderProxy_transact() [Native Bridge]

BpBinder::transact() → IPCThreadState [libbinder]

ioctl(BINDER_WRITE_READ) → Binder Driver [Kernel]

Binder 보안 모델 — Android 16

⚠ 핵심 포인트 — 반드시 숙지하세요!

- 1 커널 UID/PID 주입: 위변조 불가능한 호출자 신원 확인
- 2 SELinux binder_call: 도메인 간 Binder 호출 허용 정책
- 3 enforceCallingPermission(): 매니페스트 권한 검증
- 4 Signature Permission: 같은 서명 앱만 접근 (시스템 API 보호)
- 5 AppOps: 런타임 권한 동적 제어 + 접근 로깅



현업 트러블슈팅: Binder 보안 관련 오류

✗ **SecurityException: binder_call denied — SELinux 거부**

✓ adb logcat | grep avc:denied → audit2allow로 규칙 생성 → neverallow 충돌 확인

✗ **Permission Denial: 호출자 UID가 권한 없음**

✓ enforceCallingOrSelfPermission() 사용, AndroidManifest에 permission 선언 확인

✗ **Native Binder에서 checkPermission 누락 — 보안 취약점**

✓ IPCThreadState::self()->getCallingUid()로 반드시 호출자 확인

✗ **clearCallingIdentity 미호출 — 잘못된 권한으로 시스템 API 호출**

✓ 시스템 서비스에서 다른 서비스 호출 시 반드시 clear/restore 패턴 적용

Chapter 5 연습문제

Q1 Java BinderProxy.transact()에서 커널까지 전체 경로를 설명하시오.

Q2 SELinux binder_call 거부 시 디버깅 절차를 단계별로 설명하시오.

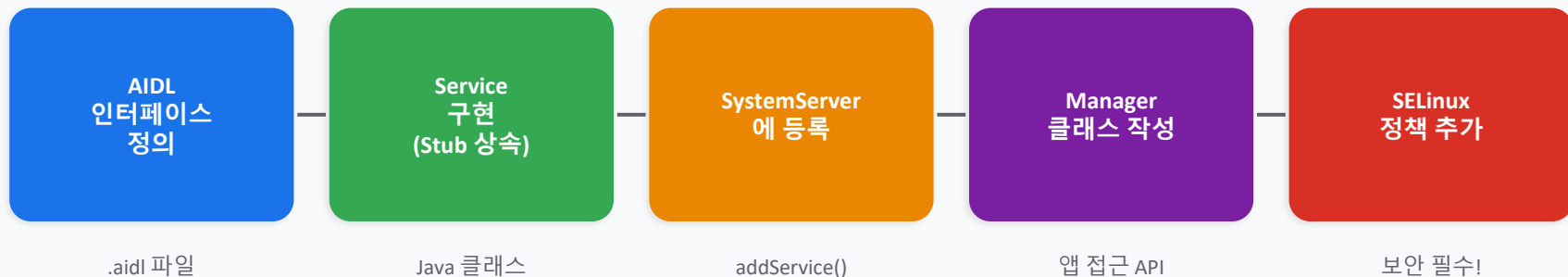
Q3 Native Binder와 Java Binder의 성능 차이와 선택 기준을 설명하시오.

CHAPTER 6

커스텀 System Service 추가

AOSP에 시스템 서비스를 추가하는 전체 과정

System Service 추가 전체 흐름



💡 Android 16: SystemServiceRegistry에 Manager 등록 → Context.getSystemService()로 접근

System Service 핵심 코드 (요약)

```
// ❶ AIDL 정의
interface IMyService { String getInfo(); }

// ❷ 서비스 구현 (★ 보안 패턴)
public class MyService extends IMyService.Stub {
    public String getInfo() {
        mContext.enforceCallingPermission(PERM, TAG);
        long token = Binder.clearCallingIdentity();
        try { return Build.MODEL; }
        finally { Binder.restoreCallingIdentity(token); }
    }
}

// ❸ SystemServer 등록
ServiceManager.addService("my_svc", new MyService(ctx));

// ❹ SELinux: service_contexts + sepolicy
```

💡 변경 파일: .aidl, Service.java, SystemServer.java, Manager.java, Registry, SELinux (6개+)

System Service 스레드 안전성

- SystemServer 내 서비스는 멀티 Binder Thread에서 동시 호출됨
- synchronized 블록 또는 ReentrantLock으로 공유 상태 보호
- Handler를 통한 메인 스레드 직렬화 패턴
- ContentObserver: 설정 변경 감지 시 Handler 스레드에서 처리
- ⚠ **Binder Thread에서 긴 작업 → 전체 시스템 ANR 위험!**
- 해결: 작업을 별도 HandlerThread로 위임, 결과를 콜백으로 전달



현업 트러블슈팅: System Service 개발 오류

✗ ServiceManager.addService() 시 SELinux 거부

✓ system/sepolicy/private/service_contexts에 서비스 등록, .te 파일에 add 규칙 추가

✗ 빌드 시 AIDL 'unknown type' 에러

✓ Android.bp의 filegroup에 .aidl 파일 추가, 패키지 경로 정확히 일치 확인

✗ System Service에서 Watchdog ANR (60초 초과)

✓ 서비스 초기화 로직 최적화, 무거운 작업은 별도 스레드, Watchdog monitor 등록 검토

✗ Manager 클래스에서 getSystemService() 반환 null

✓ SystemServiceRegistry에 REGISTER 코드 추가, Context.SYSTEM_SERVICE_NAMES에 등록



실습: 커스텀 System Service 추가 (AOSP)

1. `IMyCustomService.aidl` 작성 및 `Android.bp` 등록
2. `MyCustomService` 구현 (권한 체크 + synchronized)
3. `SystemService.java`에 `addService()` 추가
4. `MyCustomManager` + `SystemServiceRegistry` 등록
5. SELinux: `service_contexts` + `.te` 파일 작성
6. 빌드 → 에뮬레이터 flash → `adb shell service call` 테스트
7. `dumpsys my_custom`으로 서비스 상태 확인

Chapter 6 연습문제

Q1 System Service 추가 시 변경해야 하는 모든 파일을 나열하시오.

Q2 Binder Thread에서 긴 작업 수행 시 발생하는 문제와 해결 패턴을 설명하시오.

Q3 Watchdog ANR이 발생하는 원인과 디버깅 방법을 설명하시오.

DAY 3

Framework Core & Overlay & Jetpack

8시간 | Chapter 7~10

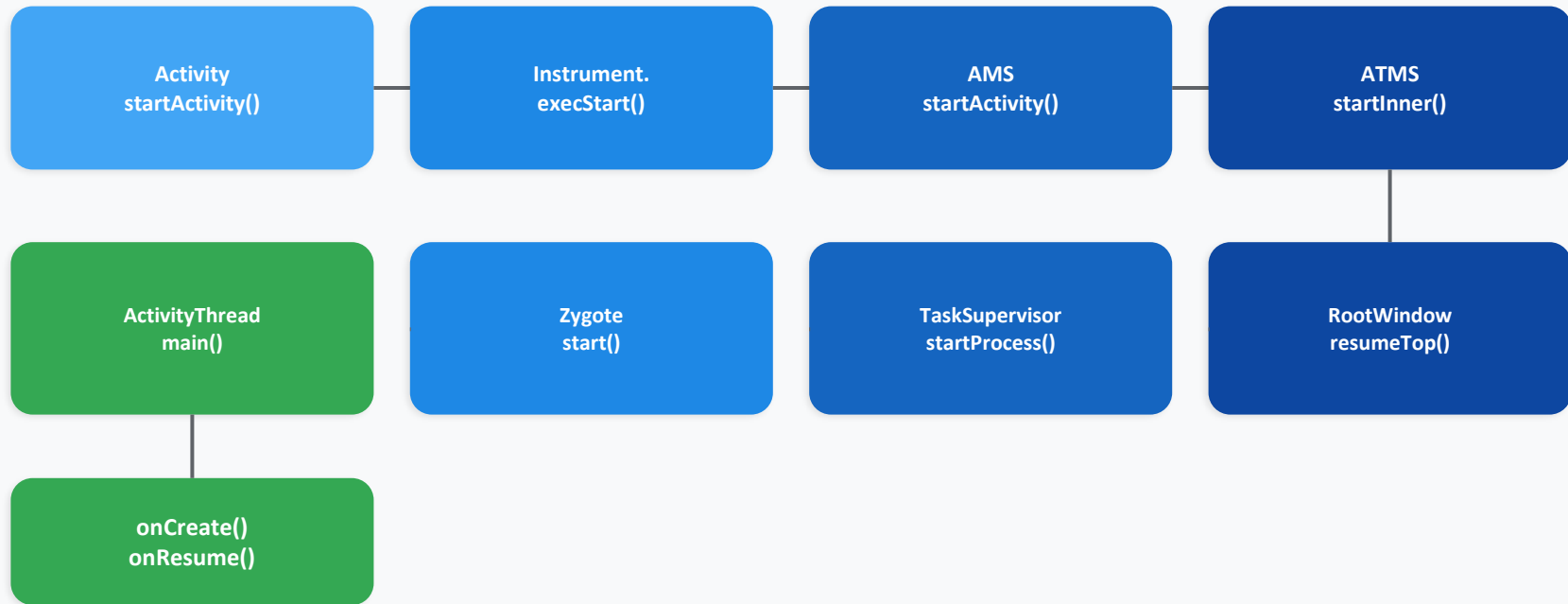


CHAPTER 7

Android Framework Core 분석

AMS, PMS, WMS 핵심 서비스 내부 동작

startActivity() 전체 흐름



△ 새 프로세스 필요 시: ATMS → AMS → ZygoteProcess.start() → fork() → ActivityThread.main()

핵심 System Service 역할

AMS

프로세스 생명주기
OomAdjuster
Broadcast 분배
ANR 감지

ATMS

Activity/Task 관리
Back Stack
Recent Tasks
Multi-Window

PMS

APK 설치/관리
Intent Resolution
권한 관리
Staged Install

WMS

윈도우 레이아웃
Input 분배
SurfaceFlinger 협력
Shell Transitions

OomAdjuster – 프로세스 우선순위

- FOREGROUND_APP (0): 현재 사용 → 절대 종료 안 함
- VISIBLE_APP (100): 보이지만 포커스 없음
- SERVICE (500): 서비스 실행 중
- CACHED (900+): 백그라운드 → 메모리 부족 시 우선 종료
- **Android 16: 프로세스 고정(pinning) API + Cached 앱 프리징 확대**
- OomAdjuster가 계산 실패 → LMK가 중요 프로세스 종료 가능



현업 트러블슈팅: Framework Core 오류

✗ ANR (Application Not Responding) — 5초 이내 Input 미처리

✓ /data/anr/traces.txt 분석, adb bugreport → Binder 대기 스레드 확인

✗ Activity 시작 실패 — 'Unable to find explicit activity class'

✓ AndroidManifest.xml에 Activity 선언 확인, exported/intent-filter 검토

✗ Process killed unexpectedly — OOM Killer

✓ dumsys meminfo로 메모리 상태 확인, oom_adj 값 분석

✗ Intent Resolution 실패 — 암시적 Intent에 대한 컴포넌트 없음

✓ adb shell pm query-activities --brief <intent> 로 매칭 확인



디버깅 가이드: Framework Core 디버깅

```
# ANR traces 분석
$ adb pull /data/anr/traces.txt
$ adb bugreport > bugreport.zip

# 프로세스 OOM adj 확인
$ adb shell dumpsys activity processes | grep -A 5 "oom adj"
$ adb shell cat /proc/<pid>/oom_score_adj

# Activity 스택 확인
$ adb shell dumpsys activity activities | grep -E "Task|Act"

# Intent Resolution 디버깅
$ adb shell pm query-activities -a android.intent.action.VIEW
$ adb shell dumpsys package resolvers

# WMS 윈도우 상태
$ adb shell dumpsys window windows | grep -E "Window|mFocus"
```



bugreport: 가장 포괄적인 디버깅 도구 — ANR, crash, 시스템 상태를 한 파일로 수집

Chapter 7 연습문제

Q1 startActivity() 전체 호출 체인에서 프로세스 생성이 관여하는 시점을 설명하시오.

Q2 ANR 발생 시 traces.txt에서 확인해야 할 핵심 정보 3가지를 설명하시오.

Q3 OomAdjuster의 프로세스 우선순위 계산 기준을 5가지 이상 나열하시오.

CHAPTER 8

RRO (Runtime Resource Overlays)

런타임 리소스 교체 메커니즘

RRO 동작 원리

타겟 앱 (Settings)

```
primary_color = #000  
accent_color = #F00
```

오버레이 APK

```
primary_color = #1A73E8  
accent_color = #34A853
```

OMS
(Overlay
Manager
Service)

최종 결과

```
#1A73E8 ✓  
#34A853 ✓
```

RRO 오버레이 핵심 구조

XML/Shell

```
<overlay android:targetPackage="com.android.settings"
    android:isStatic="false" android:priority="1" />
<application android:hasCode="false" />
```

```
<!-- 교체할 리소스 -->
<color name="primary_color">#1A73E8</color>
```

```
# 관리
$ adb shell cmd overlay list
$ adb shell cmd overlay enable <pkg>
$ adb shell cmd overlay disable <pkg>
```

💡 Fabricated Overlays (12+): 코드에서 동적 오버레이 생성 — `OverlayManager.commit()`



현업 트러블슈팅: RRO 오버레이 오류

✗ 오버레이 적용 안됨 — 설치는 되지만 리소스 변경 없음

✓ targetPackage/targetName 정확히 일치 확인, cmd overlay enable 후 앱 재시작

✗ 오버레이 충돌 — 여러 오버레이 간 우선순위 문제

✓ cmd overlay set-priority로 순서 조정, dumpsys overlay로 상태 확인

✗ 부팅 시 RRO 로드 실패 — 서명 불일치

✓ 시스템 오버레이는 platform 키로 서명 필요, AOSP 빌드 시 LOCAL_CERTIFICATE 설정

Chapter 8 연습문제

Q1 RRO의 동작 원리와 리소스 교체 메커니즘을 설명하시오.

Q2 정적 RRO와 동적 RRO의 차이를 설명하시오.

CHAPTER 9

SRO (Static Resource Overlays)

빌드 타임 정적 리소스 오버레이

SRO vs RRO 비교

SRO (Static)

- 빌드 시 리소스 병합
- 별도 APK 없음
- 런타임 변경 불가
- PRODUCT_PACKAGE_OVERLAYS
- OEM 디바이스 커스터마이징

RRO (Runtime)

- 런타임 리소스 교체
- 독립 오버레이 APK
- 활성화/비활성화 가능
- OverlayManagerService
- 테마, Material You



현업 트러블슈팅: SRO 오류

✗ SRO 적용 안됨 — 빌드 후 리소스 변경 없음

✓ overlay 디렉토리 경로/패키지 구조 정확히 확인, aapt2 dump resources로 병합 검증

✗ SRO + RRO 우선순위 혼동 — 예상과 다른 리소스 적용

✓ RRO가 SRO보다 우선 — 런타임 오버레이가 빌드 타임 오버레이를 덮어씀

✗ PRODUCT vs DEVICE OVERLAYS 충돌

✓ PRODUCT > DEVICE 우선순위, 동일 리소스를 양쪽에 정의하지 않도록 관리

Chapter 9 연습문제

Q1 PRODUCT_PACKAGE_OVERLAYS와 DEVICE_PACKAGE_OVERLAYS 우선순위를 설명하시오.

Q2 SRO+RRO 동시 사용 시 최종 적용 우선순위를 설명하시오.

Jetpack 라이브러리 개요 및 활용

시스템 개발자 관점 Jetpack과 Framework 연동

Jetpack 핵심 컴포넌트

Lifecycle

생명주기 인식 컴포넌트
LifecycleOwner/Observer
시스템 상태 연동

ViewModel + Flow

UI 데이터 보존
Coroutine 반응형
StateFlow 전환

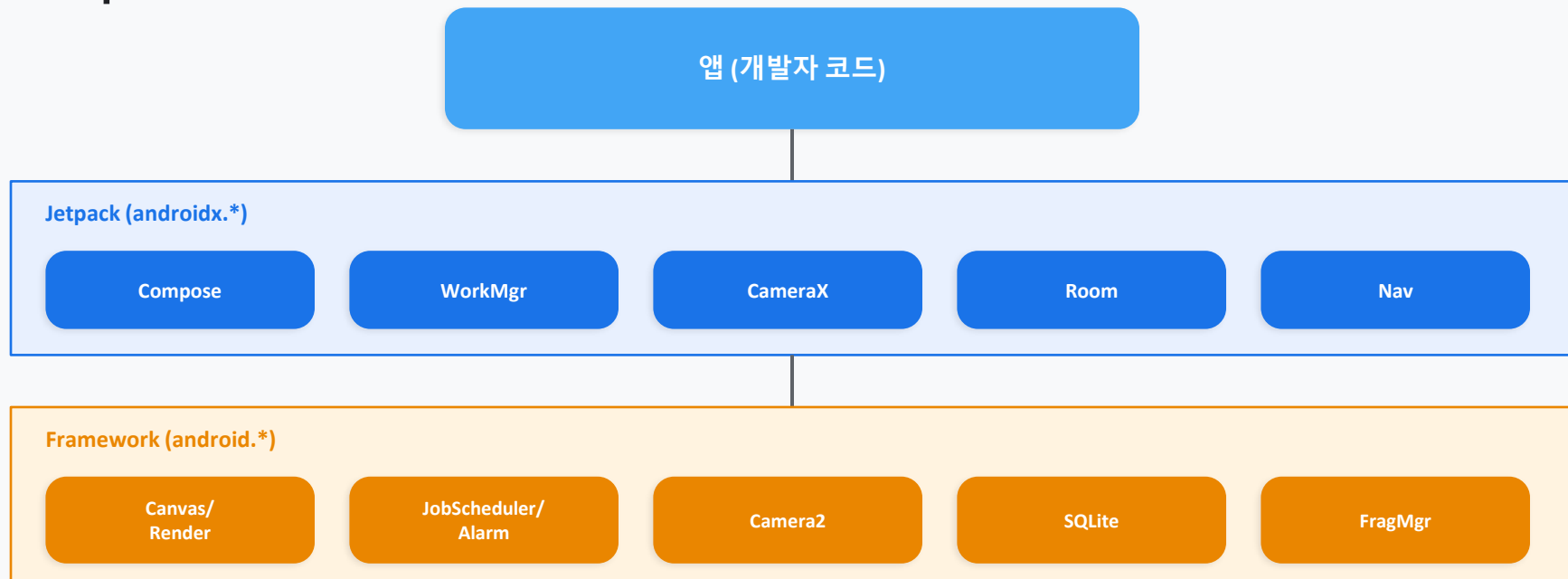
WorkManager

백그라운드 작업 스케줄링
JobScheduler + AlarmMgr
Constraints 기반

Compose

선언적 UI
Canvas/RenderNode
SystemUI 도입 (13+)
성능 최적화

Jetpack과 Framework 관계



💡 Jetpack은 Framework 위의 추상화 — 하위 호환 + 최신 패턴 제공, 내부적으로 Framework 호출



현업 트러블슈팅: Jetpack 연동 오류

✗ WorkManager 작업이 실행되지 않음 — Constraint 미충족

✓ dumpsys jobscheduler로 예약 상태 확인, 에뮬레이터에서 충전/Wi-Fi 시뮬레이션

✗ Compose Recomposition 무한 루프 — UI 프리징

✓ Layout Inspector에서 recomposition count 확인, remember/derivedStateOf 적용

✗ Room DB 마이그레이션 실패 — IllegalStateException

✓ Migration 객체에 모든 ALTER TABLE 구문 정의, fallbackToDestructiveMigration() 테스트용

✗ Lifecycle Observer 메모리 누수 — Activity 회전 시

✓ LifecycleOwner에 바인딩, viewLifecycleOwner 사용 (Fragment에서)

Chapter 10 연습문제

Q1 Compose Recomposition과 Framework 렌더링 연동을 설명하시오.

Q2 WorkManager가 내부적으로 사용하는 Framework API를 설명하시오.

Q3 SystemUI에서 Compose 도입의 이점과 과제를 논하시오.

DAY 4

System UI & APEX 모듈 업데이트

8시간 | Chapter 11~13

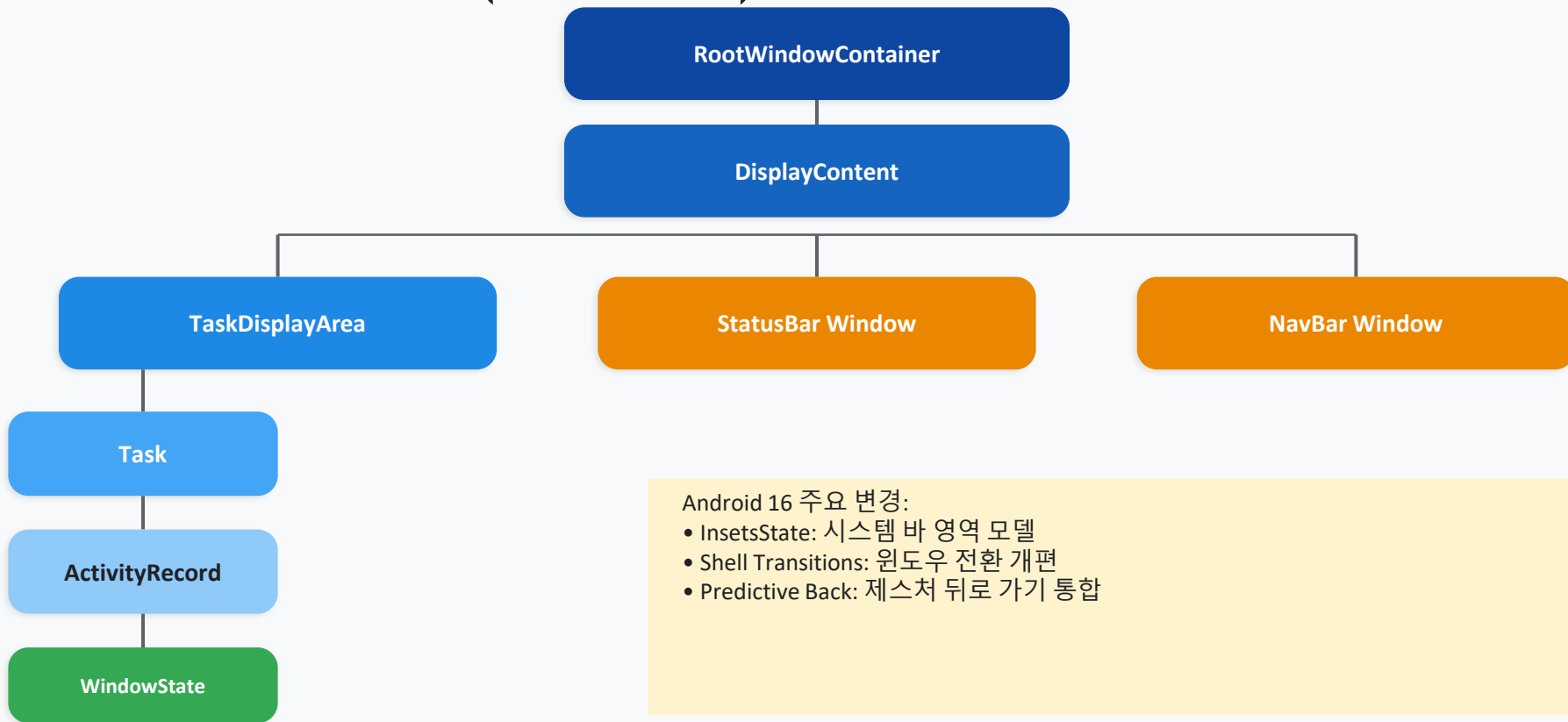


CHAPTER 11

WindowManager 및 System UI 구조

WMS와 SystemUI 내부 아키텍처

Window 계층 구조 (Android 16)



SystemUI 핵심 컴포넌트

Status Bar

시계, 배터리, 신호세기
Notification 아이콘
IconController

Quick Settings

QSTileHost 관리
QSTile 상태/UI
TileService API
config_quickSettingsTiles

Notification

StackScrollLayout
ExpandableRow
Heads-Up Display
Ranking 알고리즘

Keyguard

KeyguardViewMediator
Bouncer (PIN/패턴)
BiometricUnlock
AOD



현업 트러블슈팅: WindowManager & SystemUI 오류

✗ 윈도우 누수 (Window Leak) — Activity 종료 후 Dialog 잔존

✓ onDestroy()에서 dialog.dismiss() 호출, isFinishing() 체크 후 show()

✗ SystemUI crash loop — 상태바/네비바 사라짐

✓ logcat -s SystemUI로 crash trace 확인, RRO 충돌 여부 cmd overlay list 체크

✗ Input Dispatching 지연 — 터치 5초+ ANR

✓ dumpsys input → pending events 확인, WMS focus window 상태 분석

✗ Insets 계산 오류 — 앱 UI가 상태바/네비바와 겹침

✓ WindowInsetsCompat API 사용, setDecorFitsSystemWindows(false) 확인



디버깅 가이드: WMS & SystemUI 디버깅

```
# 윈도우 목록 및 포커스
$ adb shell dumpsys window windows | head -50
$ adb shell dumpsys window | grep mCurrentFocus

# SystemUI 상태
$ adb shell dumpsys statusbar
$ adb shell dumpsys notification

# Input 상태 (터치/키 이벤트)
$ adb shell dumpsys input

# SystemUI 강제 재시작
$ adb shell am crash com.android.systemui

# Layout Inspector (Android Studio)
# → 에뮬레이터에서 실시간 UI 계층 분석
```



Window Leak 디버깅: StrictMode + detectLeakedClosableObjects() 활성화

Chapter 11 연습문제

Q1 Window 계층의 DisplayContent → Task → ActivityRecord → WindowState 관계를 설명하시오.

Q2 윈도우 누수(Window Leak)의 원인과 디버깅 절차를 설명하시오.

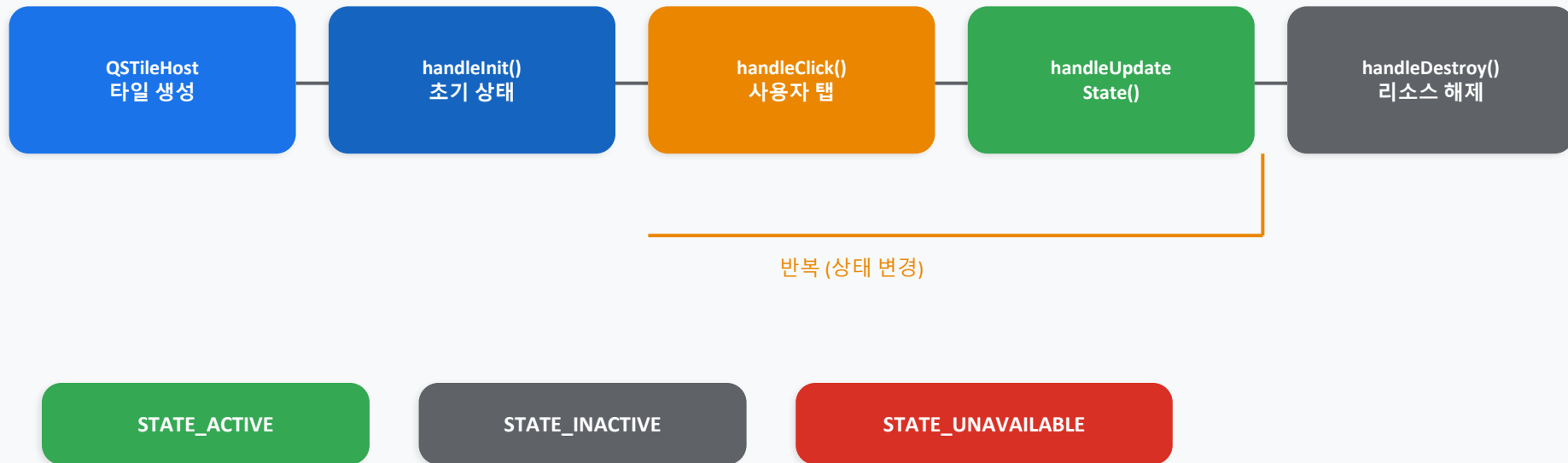
Q3 InsetsState 모델의 동작 원리를 설명하시오.

CHAPTER 12

System UI 커스터마이징

QS 타일, 상태바, 잠금화면 수정

QS 타일 생명주기



커스텀 QS 타일 핵심 메서드

```
protected void handleClick(View view) {  
    boolean newState = !mState.value;  
    Settings.System.putInt(mContext.getContentResolver(),  
        "custom_feature", newState ? 1 : 0);  
    refreshState(); // ★ handleUpdateState 트리거  
}  
  
protected void handleUpdateState(BooleanState state, Object arg) {  
    state.value = Settings.System.getInt(...) == 1;  
    state.label = "Custom";  
    state.state = state.value  
        ? Tile.STATE_ACTIVE : Tile.STATE_INACTIVE;  
}
```

💡 QsFactoryImpl에 타일 키 등록 + config_quickSettingsTiles에 추가 필요

SystemUI 수정 방식 비교

소스 수정

- AOSP 소스 직접 수정
- QS/StatusBar/Keyguard 커스텀
- 무제한 자유도
- 빌드/유지보수 비용 높음

RRO 오버레이

- RRO 오버레이로 리소스만 교체
- 색상/아이콘/문자열 변경
- 소스 수정 없이 적용
- 레이아웃/로직 변경 불가



현업 트러블슈팅: SystemUI 커스터마이징 오류

✗ QS 타일 아이콘 표시 안됨 — null icon state

✓ handleUpdateState()에서 state.icon을 반드시 설정, ResourceIcon.get() 반환값 확인

✗ SystemUI 빌드 후 교체 시 부팅 루프

✓ 서명 불일치 — platform 키로 서명, adb sideload 대신 AOSP 빌드 통합

✗ RRO로 SystemUI 색상 변경 후 다크모드에서 깨짐

✓ night/colors.xml도 함께 오버레이, DayNight 테마 리소스 세트 완성

✗ Keyguard 커스텀 후 생체인증 작동 안됨

✓ BiometricUnlockController 연동 확인, KeyguardUpdateMonitor 이벤트 체인 검증



실습: SystemUI 커스터마이징 실습

1. adb shell dumpsys statusbar로 현재 상태 확인
2. RRO로 SystemUI 색상 변경 (오버레이 APK → 설치 → 활성화)
3. AOSP SystemUI 빌드: 커스텀 QS 타일 추가
4. 상태바에 커스텀 아이콘 (배터리 온도) 추가
5. Perfetto로 SystemUI 렌더링 성능 프로파일링

Chapter 12 연습문제

Q1 QSTileImpl 상태 관리와 refreshState() 동작을 설명하시오.

Q2 RRO 다크모드 대응 시 필요한 리소스 파일을 모두 나열하시오.

Q3 SystemUI 부팅 루프 발생 시 복구 절차를 설명하시오.

CHAPTER 13

APEX 및 모듈 업데이트

Project Mainline과 APEX 기반 시스템 모듈화

APEX 패키지 구조

APEX File (.apex)

apex_manifest
.json

AndroidManifest
.xml

apex_pubkey
(공개 키)

apex_payload.img (ext4)

Native
Libs (.so)

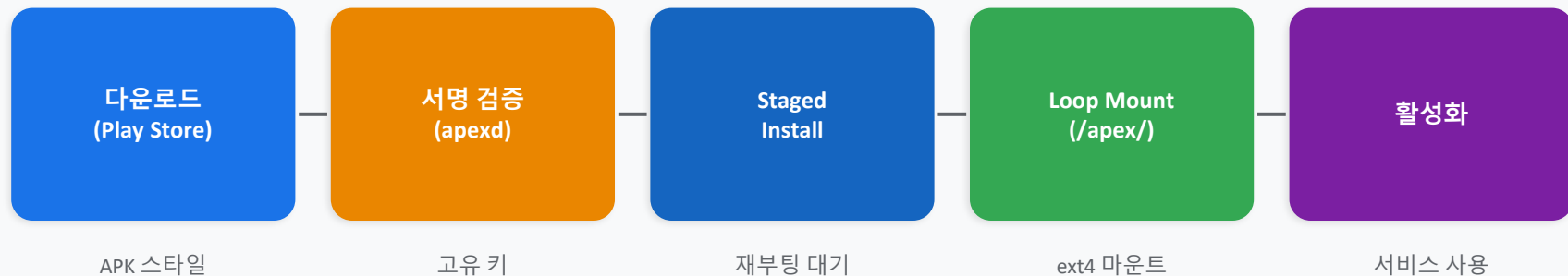
Java
Libs (.jar)

Config
Files

Binaries

dm-verity 무결성 보장

APEX 설치 및 활성화



💡 실패 시 자동 롤백 — Pre-installed: /system/apex/ | Updated: /data/apex/active/

Project Mainline — Android 16 모듈

⚠ 핵심 포인트 — 반드시 숙지하세요!

- 1 ART Runtime: APEX 독립 업데이트
- 2 Networking: DNS Resolver, Captive Portal — 보안 패치 즉시 적용
- 3 Media Components: Codec/Extractor — 취약점 빠른 대응
- 4 Conscrypt (TLS/SSL): 암호화 라이브러리 독립 업데이트
- 5 한계: 커널 모듈은 APEX 불가 (GKI 별도 관리)



현업 트러블슈팅: APEX 오류

✗ APEX 설치 실패 — 서명 검증 에러

✓ apex_pubkey와 signing certificate 일치 확인, 테스트 키로 서명 시 에뮬레이터만 가능

✗ APEX 활성화 후 부팅 실패 — 롤백 발생

✓ adb logcat -s apexd로 실패 원인 확인, /data/apex/active/에서 문제 APEX 수동 제거

✗ APEX 모듈 간 의존성 충돌 — 라이브러리 버전 불일치

✓ apex_manifest.json의 requireNativeLibs 확인, 공유 라이브러리 버전 호환성 검증

✗ Staged Install 후 재부팅해도 적용 안됨

✓ cmd apex list에서 active version 확인, staged session 상태: pm get-stagedsessions



디버깅 가이드: APEX 관리 및 디버깅

```
# APEX 목록
$ adb shell pm list packages --apex-only
$ adb shell cmd apex list

# 상세 정보
$ adb shell cmd apex info com.android.art

# 활성화 상태
$ ls /apex/
$ adb shell cmd apex active-list

# Staged session 확인
$ adb shell pm get-stagedsessions

# APEX 설치 (staged)
$ adb install --staged com.android.media.apex

# apexd 로그 확인
$ adb logcat -s apexd

# APEX 롤백 (factory reset)
$ adb shell cmd apex rollback <session_id>
```



에뮬레이터에서 /apex/ 디렉토리 확인: 마운트된 APEX 모듈 목록 및 버전 확인 가능

Chapter 13 연습문제

Q1 APEX와 APK의 구조적 차이와 설치 메커니즘을 설명하시오.

Q2 apexd의 롤백 메커니즘을 단계별로 설명하시오.

Q3 APEX 설치 실패 시 디버깅 절차를 설명하시오.

Q4 Project Mainline이 보안 업데이트에 미치는 영향을 논하시오.

APPENDIX

부록: 필수 명령어 모음 & 참고 자료



디버깅 가이드: 필수 ADB 명령어 모음 (1)

=== 프로세스 & 메모리 ===

\$ adb shell ps -A | grep <keyword>

\$ adb shell dumpsys meminfo <package>

\$ adb shell dumpsys activity processes

\$ adb shell cat /proc/<pid>/oom_score_adj

프로세스 검색

메모리 상세

프로세스 OOM 상태

OOM 점수

=== Activity & Service ===

\$ adb shell dumpsys activity activities

\$ adb shell dumpsys activity services

\$ adb shell dumpsys activity providers

Activity 스택

실행 중 서비스

Content Provider

=== 패키지 ===

\$ adb shell pm list packages -f

\$ adb shell pm dump <package>

\$ adb shell pm query-activities -a <action>

설치 앱 목록+경로

패키지 상세 정보

Intent 매칭



디버깅 가이드: 필수 ADB 명령어 모음 (2)

```
# === Binder & IPC ===
$ adb shell service list           # 시스템 서비스 목록
$ adb shell service call <svc> <code> # 서비스 직접 호출
$ adb shell dumpsys <service_name>  # 서비스 상태 덤프

# === 시스템 UI ===
$ adb shell dumpsys statusbar      # 상태바 상태
$ adb shell dumpsys notification  # 알림 목록
$ adb shell cmd overlay list       # RRO 오버레이

# === 성능 분석 ===
$ adb shell dumpsys gfxinfo <package> # 렌더링 성능
$ adb shell dumpsys batterystats    # 배터리 사용량
$ adb bugreport > bugreport.zip      # 종합 리포트
```



bugreport: ANR, crash, 시스템 상태를 한 파일로 — 현업 이슈 보고 시 필수 첨부



디버깅 가이드: Perfetto 트레이싱 가이드

```
# 기본 시스템 트레이스 수집 (10초)
$ adb shell perfetto \
  -c - --txt \
  -o /data/misc/perfetto-traces/trace \
  <<EOF
buffers: { size_kb: 65536 }
duration_ms: 10000
data_sources: { config {
  name: "linux.ftrace"
  ftrace_config {
    ftrace_events: "sched/sched_switch"
    ftrace_events: "binder/binder_transaction"
    ftrace_events: "power/cpu_frequency"
  }
}}
data_sources: { config { name: "linux.process_stats" }}
EOF

# 트레이스 파일 가져오기
$ adb pull /data/misc/perfetto-traces/trace trace.pb
# ui.perfetto.dev 에서 열기
```



Perfetto UI: <https://ui.perfetto.dev> — 브라우저에서 .pb 파일을 드래그&드롭으로 분석

현업 디버깅 체크리스트

- 1단계: 로그 수집 — `adb logcat -b all > full_log.txt`
- 관련 TAG 필터링: `logcat -s ActivityManager,SystemServer,Binder`
- 2단계: 시스템 상태 덤프 — `adb bugreport`
- ANR traces: `/data/anr/traces.txt`, tombstone: `/data/tombstones/`
- 3단계: 재현 조건 격리 — 최소 재현 시나리오 작성
- 특정 API 레벨, 특정 디바이스, 특정 시퀀스에서만 발생하는지 확인
- 4단계: 트레이스 분석 — Perfetto로 시스템 레벨 프로파일링
- CPU, Binder, 메모리, 렌더링 병목 구간 시각적 확인
- 5단계: 수정 → 검증 → 회귀 테스트

추천 참고 자료

AOSP 소스

android.googlesource.com
cs.android.com (검색)
Android 16 브랜치 참조
실제 코드로 학습

공식 문서

developer.android.com
Platform Architecture
AIDL / Binder 가이드
Jetpack 라이브러리

디버깅 도구

Perfetto: ui.perfetto.dev
Android Studio Profiler
Systrace (레거시)
Layout Inspector

커뮤니티

Android Building (메일링)
Stack Overflow
XDA Developers
AOSP Gerrit Code Review

교육 완료!

4일간의 Android Framework 16 심화 교육을 마칩니다

- Day 1: Android 16 아키텍처 & Binder IPC 기초/실습
- Day 2: Binder 심화 분석 & 커스텀 System Service
- Day 3: Framework Core & Resource Overlays & Jetpack
- Day 4: System UI 커스터마이징 & APEX 모듈 업데이트

실습 환경: Windows 11 + Android Studio + Emulator (API 36)